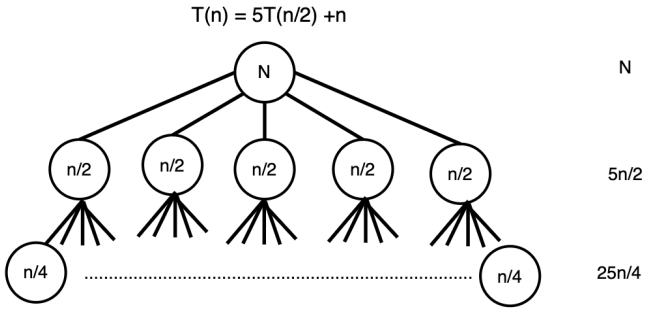
# DSAII- HW4

Tomas Salaz

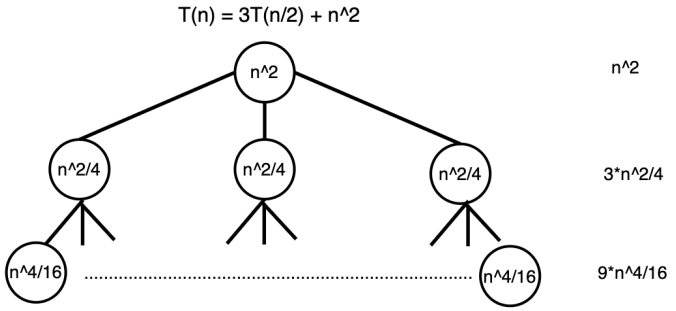
Februrary 23, 2026

## Problem 1

Given $T(n) = 5T(n/2) + n$ and $T(n) = 3T(n/2) + n^2$, $T(c) = \Theta(1)$ for any constant c. Which has better asymptotic cost, solve using recurrence trees.




For the 1st algorithm we can see that
$T(n) = \sum_{k=0}^{log_2 n-1} 5^k \cdot n/2^k = \sum_{k=0}^{log_2 n-1} n(5/2)^k$.
We need to add the leaf nodes to the sum so we can count them, $n/2^k = 1$, when we solve for k we get $k = log_2 n$. The number of leaves is $5^k = 5^{log_2 n}$, but since $T(1) = \Theta(1)$ the contribution is $5^{log_2 n} = n^{log_2 5}$, Thus
$T(n) = \sum_{k=0}^{log_2 n-1} n(5/2)^k + n^{log_2 5}$
The term $5^{log_2 n}$ is the dominant term in the sum since it is geometric and $5/2 > 1$, this $T(n) = \Theta(n^{log_2 5}) \approx \Theta(n^{2.32})$




For the 2nd algorithm we can see that
$T(n) = \sum_{k=0}^{log_2 n-1} 3^k \cdot n^2/4^k = \sum_{k=0}^{log_2 n-1} n^2(3/4)^k$
We need to add the leaf nodes, so we get something similar. $k = log_2 n$, so number of leafs $= 3^{log_2 n} = n^{log_2 3}$. Thus
$T(n) = \sum_{k=0}^{log_2 n-1} n^2(3/4)^k + n^{log_2 3}$// Since $3/4 < 1$ the geometric series converges to the leading coefficient, $n^2$ so this algorithm has $T(n) = \Theta(n^2)$.

From the answers, we can see that the 2nd algorithm has better asymptotic costs.

## Problem 2

### Part A

For this recurrence we have to have at least $i \geq 3$ since there are 2 base cases.
In the first case, there is $1/2$ chance the frog jumps 1 lily pad. Since we are on the $i-th$ step, we define this part as $1/2 \cdot p(i-1)$.
For the other step, there is a $1/2$ the frog jumps 2 lily pads, so similarly, we can define this part as $1/2 \cdot p(i-2)$.
All together, we get $p(i) = 1/2 \cdot p(i-1) + 1/2 \cdot p(i-2)$ for $i \geq 3$, where $p(1) = 1$ and $p(2) = 1/2$.

### Part B

Using annihilators to solve the recurrence. we can define the following recurrence where $T_i = T(i)$.
$T =< T_0, T_1, T_2, ... >$
$LT =< T_1, T_2, T_3, ... >$
$L^2T =< T_2, T_3, T_4, ... >$
We can define $p(i)$ in terms of $T(n)$ so we get
$p(i) = T(n) = 1/2 \cdot T(n-1) + 1/2 \cdot T(n-2)$
Using the annihilator from above we can define the recurrence to use annihilators
$L^2T =< 1/2 \cdot T_{n-1} + 1/2 \cdot T_{n-2} >$
$L^2T - 1/2LT - 1/2L = T(L^2 - 1/2L - 1/2) =< 0 >$
We can rewrite this as $L^2 - 1/2L - 1/2 = 2L^2 - L - 1 =< 0 >$

We can see that $R_1 = 1$ and $R_2 = 1/2$ because $2L^2 - L - 1 = (2L + 1)(L - 1)$
From the lookup table, we can see the general solution, in terms of $p(i)$ is
$p(i) = C_1 \cdot 1^i + C_2 \cdot (-1/2)^i$

**Part C**
Using the base cases where $P(1) = 1$ and $p(2) = 1/2$, we can start by substituting with $i = 1$
$P(1) = C_1 + C_2(-1/2)^1 = C_1 - C_2/2 = 1 \rightarrow C_1 = 1 + C_2/2$, We can use this definition of $C_1$ to find $C_2$
$P(2) = (1 + C_2/2) + C_2(-1/2)^2 = (1 + C_2/2) + C_2/4 = 1/2$
$1/2 = 1 + C_2/2 + C_2/4 \rightarrow 2/4 = 4/4 + 2C_2/4 + C_2/4 \rightarrow 2 = 4 + 2C_2 + C_2$
$2 = 4 + 2C_2 + C_2 \rightarrow -2 = C_2(2 + 1) \rightarrow C_2 = -2/3$
Using $C_2$ we can find $C_1$ in the original $P(1)$
$P(1) = 1 = C_1 + (-2/3)(-1/2) \rightarrow 1 = C_1 + 1/3 \rightarrow C_1 = 2/3$
So the exact solution for the recurrence is
$p(i) = 2/3 \cdot [1 - (-1/2)^i]$

**Part D**
Let $X_i$ be the indicator RV for the frog visiting pad $i$. Then $X = \sum_{i=2}^{n} X_i$.
Then expectation by LOE is $E[X] = \sum_{i=2}^{n} E[X_i] = \sum_{i=2}^{n} p(i)$
Contnuing we get
$\sum_{i=2}^{n} p(i) = \sum_{i=2}^{n} 2/3 \cdot [1 - (-1/2)^i] = 2/3(n - 1) - 2/3 \sum_{i-2}^{n} (-1/2)^i$
This geometric sum $\sum_{i=1}^{n} r_i = 1 - r^{n-1}/1 - r$
When we substitute in our values we get
$1/4 \cdot 2/3 \cdot (1 - (-1/2)^{n-1}) = 1/6[1 - (-1/2)^{n-1}]$
Substituting back into our Expecation
$E[x] = 2/3(n - 1) - 2/3[1/6(1 - (-1/2)^{n-1}]$
$E[x] = 2/3(n - 1) - 1/9[1 - (-1/2)^{n-1}]$

## Problem 3

**Part A**
If $n$ is length of input $A$, then *SillySort*(A,1,n) correctly sorts $A[1...n]$. Let $n = j - i + 1$ and $k = \lfloor n/3 \rfloor$. Each recursive call operates on $\lceil 2n/3 \rceil$ elements.

**BC**: If $i + 1 \geq j$, Cases $n = 1, 2$, the algorithm just compares and may swap $A[i]$ with $A[j]$, which correctly sorts the elements in $A$

**IH**: SillySort correctly sorts any subbarray of size $m$, where $m < n$.

**IS**: Assuming SillySort Correctly sorts any sub array size $m < n$, by the IH
Recursion 1: SillySort(A,i,j-k) correctly sorts $A[i..j-k]$. The $n-k$ smallest elements are in this sub array and are sorted, the other $k$ elements are somewhere else unsorted
Recursion 2: SillySort(A,i+k, j) correctly sorts $A[i + k..j]$. After this call, the

$k$ largest elements are sorted and are in their final positions.
Recursion 3: SillySort(A,i,j-k). Correctly sorts the remaining $A[i..j-k]$ which is now correctly placed.
At this pointl, the entire array is sorted.

**Part B**
Each Call does $O(1)$ work and makes 3 recursive calls on a sub array, size $\lceil 2n3 \rceil$.
Thus the recurrence is
$T(n) = 3T(2n/3) + O(1)$
Using the master method we can define $a = 3, b \rightarrow n/b = 2n/3 \rightarrow b = 3/2$ and $f(n) = O(n^0) = 1$
We compare with the log
$log_{3/2}3 \approx 2.71$ so we use use case 1, thus
$T(n) = \Theta(n^{log_{3/2}3})$
**Part C**
The other algorithms runtimes, in the worst case have been defined in previous lectures and classes.
MergeSort = $\Theta(n\ log\ n)$
QuickSort = $\Theta(n^2)$
InsertionSort = $\Theta(n^2)$
Based on these times from other classes, SillySort is the worst with approximately $\Theta(n^{2.71})$ runtime in the worst case.

## Problem 4

Let $f(v)$ be the number of R's in the tree rooted at $v$. Every Node in the Right Subtree $R(v)$ gets an extra R before its name. This contributes to $s(r(v))$ additional R's since they are needed in counting.

$f(v) = 0$ if v=NULL
$f(v) = f(\ell(v)) + f(r(v)) + s(r(v))$ otherwise

where $s(NULL) = 0$
For verification on the tree in the example, The Right Subtree , $s(r)$ has 5 nodes. So we know we have atleast 5 R's. Then computing bottom up we get that $f(\ell(v))$ has 1 and $f(r(v))$ has 4 so there are 10 total.